Turning a Chessboard into a Field

My first Python project, explained from scratch

Physics, 3rd semester · Python with python-chess, NumPy, Matplotlib and Seaborn

1. What is this about?

A chessboard has 64 squares, and every piece can "reach" some of them. I wanted to see, for every square, which side reaches it more. So I wrote a short Python program that colours the board: **red** where White has more reach and **blue** where Black does.

Physicists do this kind of thing all the time: take a system, give every point in space one number, and plot it. That is called a **field**. Here the "space" is just the 8×8 board.

> **Four words you will see**
>
> - **Attack:** a piece attacks a square if it could capture something there on its next move.
> - **Net control:** White's attackers minus Black's attackers on a square (then multiplied by a centre weight).
> - **Centre weight:** a number from 1.0 at the edges up to 1.5 on the four middle squares, because the centre matters more in chess.
> - **Heatmap:** a picture where colours stand for numbers.

2. How it works

For each square, the program does the same four steps:

- **1.** Count the White pieces that attack the square.
- **2.** Count the Black pieces that attack it.
- **3.** Subtract: White − Black. Positive means White is ahead there, negative means Black is, zero means a tie or nobody reaches it.
- **4.** Multiply by the centre weight, then pick a colour.

> **A worked example you can check yourself**
>
> In the starting position, take the square **c3**. Three White pieces attack it: the knight on b1, the pawn on b2 and the pawn on d2. No Black piece does. So White − Black = 3 − 0 = 3, and the centre weight of c3 is 1.2. Net control = 3 × 1.2 = **3.6**, which is exactly the number the heatmap shows on c3.

The physics idea: pieces add up like sources

Each piece adds its own pattern to the board: positive for White, negative for Black. The patterns simply add together. Physicists call this **superposition**; it is the same reason the electric fields of two charges add up. Below is the smallest example I could make, with one white knight and one black bishop.

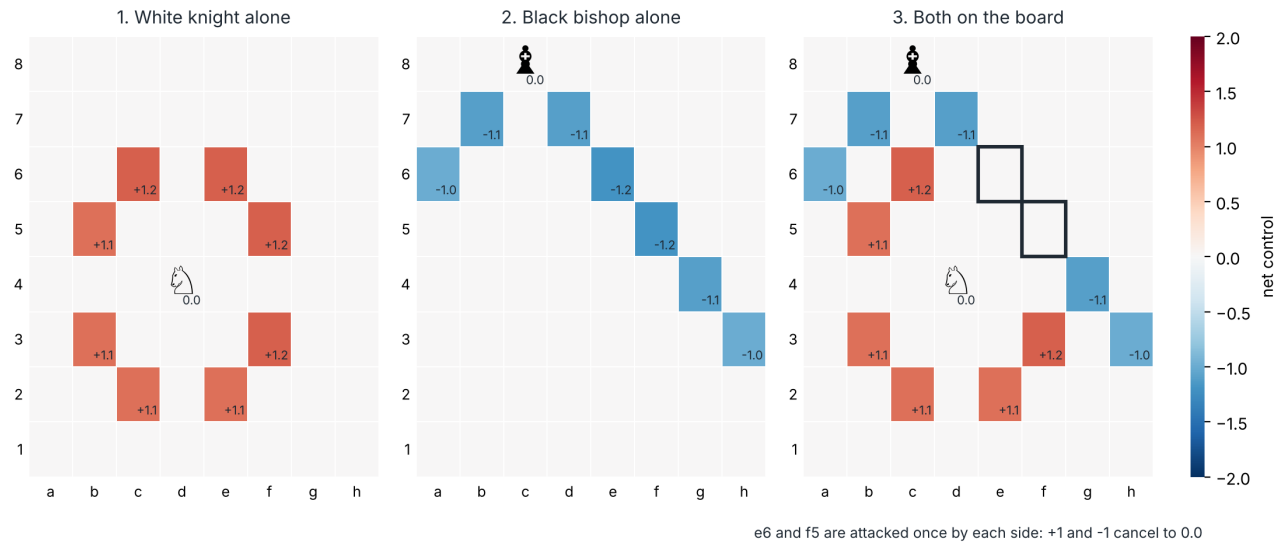


Figure 1. Left: the knight's pattern. Middle: the bishop's pattern. Right: both together. The two ringed squares are reached once by each side, so +1 and −1 cancel to 0.0.

3. The four positions I tested

I ran the program on four positions. All four pictures use the same colour scale (−6 to +6), so the same shade always means the same number. The small number in the corner of each square is its net control, and each title shows how much territory each side has (the sum of the squares where that side is ahead).

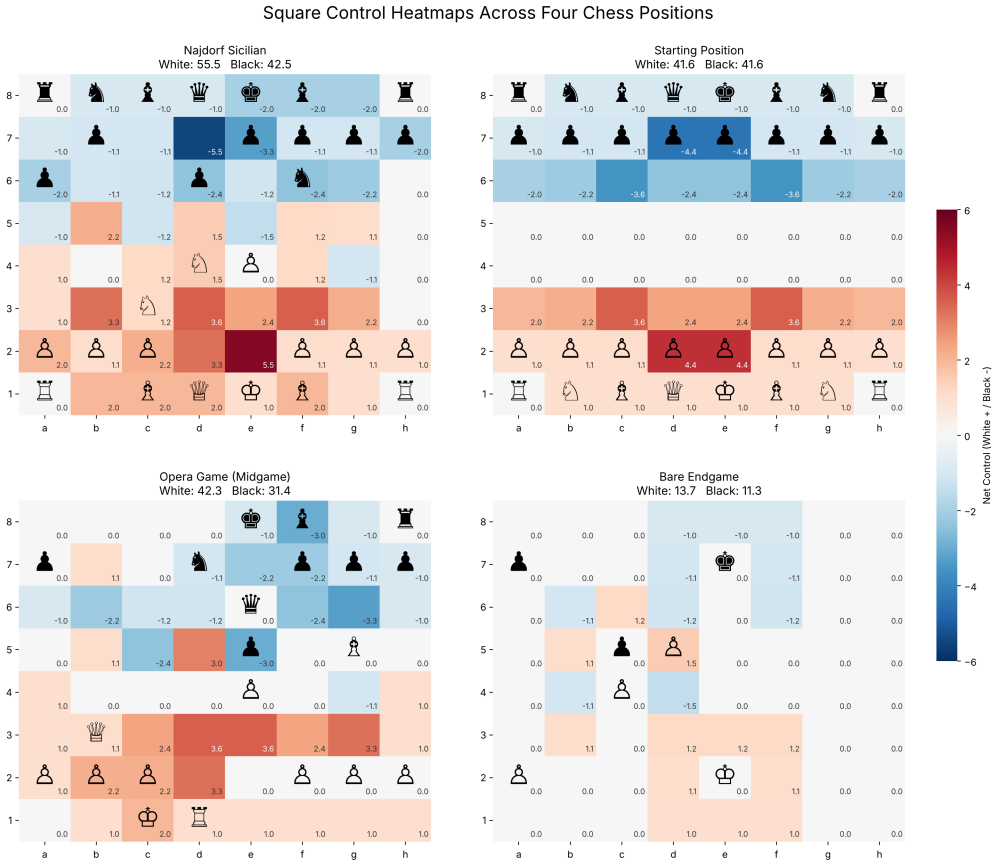


Figure 2. The starting position, the Najdorf Sicilian (an opening), the Opera Game (a middlegame), and a bare endgame.

4. What each picture shows

Position	White	Black	In one line
Starting position	41.6	41.6	Perfectly fair, and a mirror image
Najdorf Sicilian	55.5	42.5	White a bit ahead; two very hot squares
Opera Game	42.3	31.4	White ahead; the whole third rank glows red
Bare endgame	13.7	11.3	Only a few small patches near kings and pawns

Starting position. Both sides get exactly 41.6, and the picture is a mirror image. Ranks 4 and 5 are blank because nothing reaches them yet. That is what should happen in a fair, symmetric position, so it works as a first test of the code.

Najdorf Sicilian. White is a little ahead (55.5 to 42.5). The hottest squares, e2 (+5.5) and d7 (−5.5), are where five of one side's own pieces all guard the same square. That is a hint about what the number really measures (see section 5).

Opera Game. This is the position just before Paul Morphy's famous finish in 1858. White's pieces point at the middle of the board, and rank 3 is bright red.

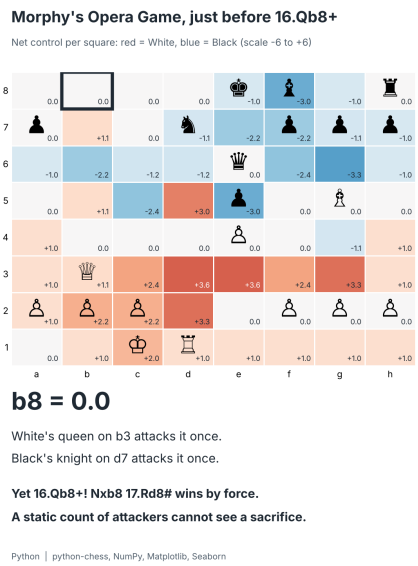
Bare endgame. Only kings and pawns are left, so only a few squares have values. Each king makes a small patch of about ±1 around itself. On e6, a white pawn and the black king both reach the square, so +1 and −1 cancel to 0.0.

5. Three things I learned

1. The code passed a sanity check. A mirror-image position should give a mirror-image map, and the starting position does (41.6 and 41.6).

2. The map shows coverage, not who is winning. A square guarded five times looks "strong" even if no enemy piece is near it, because a side's own defenders count as control.

3. A snapshot cannot see tactics. In the Opera Game, White wins by force: **16.Qb8+!** Nxb8 **17.Rd8#**. The queen is sacrificed to pull the knight away from d7, and then the rook gives checkmate. But my map shows **b8 = 0.0**: White's queen on b3 reaches b8 once, and Black's knight on d7 reaches it once. The map only sees who reaches a square *right now*, not what happens next.



> **The main lesson:** a model is most useful where it fails. The b8 square shows exactly what this model cannot do, and that tells me what to add next.

6. Where the physics fits, and where it does not

- **It fits:** the result is a **scalar field** (one number at every point of a grid), and the pieces act like **sources** whose contributions add by **superposition**.
- **It fits:** the centre weight works a bit like a shallow **potential well** in the middle, because attacks there count more.
- **It fits: symmetry** is a good test. Flip the board and swap the colours, and the starting position must look the same.
- **It does not fit:** there is no real energy, nothing is conserved, and there is no time evolution, temperature or entropy here. I call it a "field" and not a "phase space" because phase space needs momentum as well as position, and this map only has positions.

7. Limits of this project

- Every piece counts the same, so a pawn's attack equals a queen's.
- A side's own defenders are counted as control.
- Pins and hidden attacks along a line are ignored.
- It is one frozen position, with no sense of what happens next.
- I chose the centre weights by hand; they are not fitted to data.
- Four positions are an illustration, not a proper sample, so I cannot draw statistical conclusions.

8. What I want to try next

- Give each piece a value (pawn 1, knight and bishop 3, rook 5, queen 9) so a queen's attack counts more than a pawn's.
- Compute the map after every move of a whole game and plot how it changes.
- Compare the maps with a chess engine's evaluation on many real games, to see whether central control predicts anything.
- Measure whether the spread of the field (an entropy-like number) changes through a game. This is an idea to test, not a result I have.

> **Feedback is very welcome.** I am still learning, so if you work in data science, physics or game theory and see a better way to map this, I would love to hear it.